

AgroVision

09 - React Native Aplicado

Stack mobile, libs, estrutura e patterns

TCC-Docs · Material de estudo
Luiz Carlos Junior & Equipe AgroVision
Maio de 2026

1. Por que React Native e Expo

React Native permite escrever uma unica base de codigo TypeScript que roda nativamente em iOS e Android. **Expo** e um framework sobre o RN que prove um conjunto curado de APIs (camera, file system, sharing, sqlite) e um workflow de desenvolvimento sem precisar compilar Android Studio/Xcode localmente para a maior parte das tarefas.

2. Estrutura de pastas

```
src/  
  screens/ - telas (HomeScreen, ResultScreen, AuditarModeloScreen, ...)  
  components/ - componentes reutilizaveis  
    ui/ - primitivos (Button, Card, LoadingSpinner)  
    common/ - layout (SafeAreaView, Header)  
    xai/ - especificos da auditoria (HeatmapOverlay, MetricsPanel, AuditExplanation)  
  services/ - integracao externa  
    ml/ - classifyService, xaiService (chamadas WebSocket)  
    storage/ - databaseService (SQLite)  
  context/ - AnalysisContext, AppContext  
  hooks/ - useCamera, useAnalysis, ...  
  models/ - tipos TypeScript  
  navigation/ - configuracao do React Navigation  
  utils/ - constants, logger, format
```

3. Libs principais e por que estao no projeto

Lib	Versao	Funcao
expo	~52	Framework principal
react-native	0.76+	Runtime mobile
typescript	~5.x	Tipagem estatica
@react-navigation/native	~7	Navegacao entre telas
@react-navigation/native-stack	~7	Stack de navegacao (push/pop)
expo-camera	~16	Acesso a camera nativa
expo-file-system	~17	Leitura/escrita de arquivos, base64
expo-sqlite	~14	Banco de dados local
expo-sharing	~12	Share nativo (iOS/Android)
expo-image-picker	~16	Selecao de imagem da galeria
axios	~1.x	Cliente HTTP (chamadas REST de fallback)

4. Padroes adotados

4.1. Functional components + hooks

Nenhum componente de classe. Todo state usa `useState`, side effects `useEffect`, derivacoes `useMemo`, callbacks estaveis `useCallback`. Padrao moderno do React (post 2019).

4.2. Context API para estado global

`AnalysisContext` (em `src/context/AnalysisContext.tsx`) mantem o estado da analise em curso (foto, resultado, erro, loading). Implementado com `useReducer`, exposto via `Provider` envolvendo o app. Telas consomem com o hook `useAnalysisContext`.

4.3. Path aliases

`tsconfig.json` define aliases (`@components`, `@screens`, `@services`, `@utils`, `@models`) para evitar imports relativos longos. Configurado via `babel-plugin-module-resolver`.

4.4. Strict TypeScript

Modo strict ativo: `strictNullChecks`, `noImplicitAny`. Todas as interfaces de payload (`ClassifyResult`, `XAIResult`, `Analysis`) estao em `src/models/`.

5. Comunicacao com o backend

Dois servicos cuidam disso:

classifyService.ts: abre `WebSocket` em `/ws/classify`, envia imagem em base64, escuta eventos de progresso e resultado.

xaiService.ts: idem para `/ws/gradcam`. Tem variante REST de fallback.

6. Persistencia local

`databaseService` e um singleton inicializado no startup do app (em `App.tsx`). Cria as tabelas se nao existem, expoe metodos assincronos. Cada operacao de IO usa `execAsync` ou `getAllAsync` do `expo-sqlite`.